

AirWrite — Master Project Documentation (Redesigned)

Project Title

AirWrite: A Gesture-Based Touchless Input System for Cursor Control and Continuous Air Handwriting

1. Abstract

AirWrite is a webcam-based touchless Human-Computer Interaction (HCI) system that enables users to fully control their computer using only hand gestures and air-written text. The system combines real-time cursor control, gesture-based clicking, continuous air handwriting, and OS-level text injection into a unified accessibility-focused interaction platform.

Using MediaPipe hand tracking, OpenCV-based stroke rendering, heuristic segmentation, and CNN-based character recognition trained on the EMNIST dataset, AirWrite allows users to write words in the air and inject them directly into any focused application such as browsers, Notepad, VS Code, or Microsoft Word.

Unlike traditional gesture systems that only classify static gestures or isolated characters, AirWrite introduces progressive real-time air-written word recognition using dynamic stroke buffering and heuristic segmentation. The project avoids dependence on specialised hardware and operates using only a standard webcam.

The system is designed as a practical assistive technology and advanced Final Year Project (FYP) combining Computer Vision, Human-Computer Interaction, Machine Learning, and Accessibility Engineering.

2. Problem Statement

Traditional computer interaction relies heavily on physical hardware such as keyboards, mice, touchscreens, and styluses. These interaction methods become restrictive in:

- accessibility-focused environments
- touchless interaction scenarios
- smart environments
- healthcare systems
- AR/VR-inspired interaction spaces
- motor impairment use cases

Existing gesture systems mainly focus on:

- static gesture classification
- cursor movement only
- air drawing without recognition
- hardware-dependent interaction systems

Most systems fail to provide:

- continuous air handwriting
- system-level text injection
- unified cursor + typing interaction
- webcam-only accessibility

AirWrite addresses these limitations by creating a unified touchless input system capable of:

- full cursor control
 - gesture clicking
 - continuous air-written word recognition
 - OS-level typing into any application
-

3. Project Objectives

Primary Objectives

1. Build a webcam-based touchless input system.
2. Enable full system cursor control using hand gestures.
3. Implement gesture-based clicking and scrolling.
4. Enable continuous air-written word recognition.
5. Inject recognised words directly into focused applications.
6. Create a unified cursor + writing interaction pipeline.
7. Build a real-time progressive recognition system.

Secondary Objectives

1. Reduce dependence on physical input devices.
 2. Improve accessibility-focused interaction.
 3. Build a scalable HCI framework.
 4. Explore real-time handwriting segmentation.
 5. Create a browser deployment foundation for future work.
-

4. Project Motivation

The motivation behind AirWrite comes from the growing need for natural and touchless computer interaction systems.

Current gesture systems either:

- control cursors only
- require expensive hardware
- fail to support typing
- are limited to demo applications
- cannot interact with the operating system directly

AirWrite aims to create a practical real-world interaction system where users can:

- move the cursor
- click applications
- write words in air
- type into any text field

using only a webcam.

The project also explores:

- Human-Computer Interaction (HCI)
 - Accessibility Engineering
 - Computer Vision
 - Real-time AI Systems
 - Air Handwriting Recognition
 - Touchless Operating System Interaction
-

5. Existing System Analysis

Existing Technologies

1. Gesture Mouse Systems

Allow cursor movement and clicking.

Limitations:

- no handwriting recognition
- limited interaction scope

2. Air Drawing Systems

Allow users to draw in air.

Limitations:

- no OCR pipeline
- no text generation
- no OS-level integration

3. Stylus OCR Systems

Use tablets and styluses for handwriting.

Limitations:

- hardware dependency
- touchscreen requirement

4. AR/VR Interaction Systems

Support air gestures.

Limitations:

- expensive hardware
- depth sensors required
- impractical deployment

6. Proposed System

AirWrite combines:

- gesture mouse interaction
- continuous air handwriting
- word recognition
- operating system text injection

into a single unified pipeline.

The user can:

- move the cursor using finger tracking
- click using pinch gestures
- switch modes using an open palm gesture

- write complete words continuously in air
- inject recognised words into any focused application

The system recognises complete words progressively while the user writes and finalises prediction after a 1.2 second pause.

7. Core Novelty of the Project

Major Innovations

1. Unified Cursor + Air Writing System

Combines cursor control and handwriting recognition in one interaction pipeline.

2. Webcam-Only Touchless Input

No gloves, stylus, depth camera, or special hardware required.

3. OS-Level Text Injection

Recognised text is typed directly into any focused application.

4. Progressive Real-Time Word Recognition

The system updates internal predictions continuously while the user writes.

5. Heuristic Segmentation Pipeline

Continuous word strokes are segmented dynamically using motion heuristics.

6. Accessibility-Focused Interaction

Designed as a practical assistive technology.

7. Continuous Air-Written Word Pipeline

The user writes complete words fluidly before final recognition.

8. Scope of the Project

Current Scope

- Webcam-based cursor control

- Gesture clicking
- Scroll gestures
- Continuous air-written English words
- Uppercase + lowercase + digit recognition
- Real-time prediction updates
- OS-level typing
- Desktop deployment

Future Scope

- Browser deployment
 - Hindi/Devanagari support
 - Mobile deployment
 - AR/VR integration
 - Voice integration
 - Transformer-based OCR
 - Multi-user interaction
 - Smart classroom systems
 - Healthcare accessibility systems
-

9. System Architecture

High-Level Architecture

Webcam → MediaPipe Hands → Gesture Detection → Mode Switching → Stroke Buffer Engine → Heuristic Segmentation → CNN Recognition → Progressive Prediction Buffer → Final Word Commit → pyautogui Injection

10. Modes of Operation

Mode 1 — Cursor Mode

Functionalities

- Cursor movement
- Left click
- Right click
- Scroll control

Gestures

Action	Gesture
Move cursor	Index finger extended
Left click	Thumb-index pinch
Right click	Hold pinch for 1 second
Scroll	Two-finger pinch + drag
Switch mode	Open palm hold

Mode 2 — Write Mode

Functionalities

- Continuous air-written word input
- Real-time progressive recognition
- Final word injection into focused app

Gestures

Action	Gesture
Draw stroke	Pinch + finger movement
Pen up	Pinch release
Word complete	1.2 second pause
Switch mode	Open palm hold
Backspace	Keyboard key

11. Working Mechanism

Step 1 — Webcam Capture

OpenCV continuously captures webcam frames.

Step 2 — Hand Landmark Detection

MediaPipe Hands extracts 21 hand landmarks.

Step 3 — Mode Detection

The system checks whether the user is in:

- Cursor Mode
- Write Mode

based on gesture detection.

Step 4 — Cursor Interaction

In Cursor Mode:

- landmark 8 controls cursor position
- pinch gesture triggers clicks
- two-finger pinch enables scrolling

Step 5 — Stroke Rendering

In Write Mode:

- pinch activates drawing
- fingertip trajectories are recorded
- strokes are rendered on a virtual canvas

Step 6 — Continuous Stroke Buffering

All stroke points are stored dynamically:

(x, y, timestamp)

This creates a temporal stroke history for the current word.

Step 7 — Progressive Recognition

While the user writes:

- segmentation updates continuously
- candidate characters are extracted
- CNN predictions update internally
- running word prediction stabilises progressively

No text is injected yet.

Step 8 — Heuristic Segmentation

Character boundaries are estimated using:

- velocity drops
- small pauses
- stroke gaps
- x-axis spacing
- pen lifts

Step 9 — Word Completion

If:

pause > 1.2 seconds

then:

- current prediction finalises
- word is committed
- pyautogui injects the word into the focused application

Step 10 — Wipe Gesture Handling

AirWrite supports a full-hand horizontal wipe gesture for deletion and correction.

While Writing

If the user performs a wipe gesture before word commitment:

- current stroke buffer clears
- current prediction resets
- user can rewrite the word

After Word Commitment

If the user performs a wipe gesture after injection:

- previously injected word is deleted
- system simulates Ctrl + Backspace behaviour

Detection Logic

The gesture is detected using:

- open palm detection
- rapid horizontal hand movement

- minimum swipe distance threshold
- minimum x-axis velocity threshold

A short cooldown prevents repeated accidental deletions.

Step 11 — Canvas Reset

After successful word injection or wipe reset:

- stroke buffer clears
 - next word begins
-

12. Real-Time Recognition Strategy

AirWrite does not use industrial transformer-based online OCR.

Instead, it uses:

Progressive Heuristic Recognition

The system continuously updates INTERNAL predictions while the user writes.

Example:

H → HE → HEL → HELL → HELLO

However:

- predictions are not immediately typed
- the final word commits only after pause detection

This significantly reduces complexity while preserving real-time interaction feel.

13. Functional Requirements

Core Requirements

FR1 — Webcam Access

System must access webcam feed.

FR2 — Hand Tracking

System must detect and track hand landmarks.

FR3 — Cursor Control

System must control OS cursor.

FR4 — Click Detection

System must support left/right click gestures.

FR5 — Scroll Support

System must support scrolling gestures.

FR6 — Stroke Rendering

System must render air-written strokes.

FR7 — Continuous Word Buffering

System must store temporal stroke history.

FR8 — Heuristic Segmentation

System must estimate character boundaries.

FR9 — Character Recognition

System must recognise segmented characters.

FR10 — Progressive Prediction

System must update running word predictions.

FR11 — OS-Level Typing

System must inject words into focused applications.

FR12 — Wipe Gesture Detection

System must detect full-hand wipe gestures.

FR13 — Word Deletion Support

System must support clearing current word buffers and deleting previously injected words.

14. Non-Functional Requirements

Performance

- low latency
- realtime webcam processing
- smooth cursor movement

Reliability

- stable gesture recognition
- reduced jitter
- consistent prediction stability

Usability

- simple interaction model
- low learning curve
- intuitive gestures
- touchless correction workflow
- intuitive gestures

Accessibility

- touchless interaction
- assistive usability

Scalability

- future multilingual expansion
- future browser deployment

15. Hardware Requirements

Minimum

- Intel i5 Processor
- 8 GB RAM
- HD Webcam
- Windows/Linux OS

Recommended

- Intel i7 / Ryzen 7
 - 16 GB RAM
 - Dedicated GPU optional
 - High FPS webcam
-

16. Software Requirements

Development Tools

- Python 3.11
- VS Code
- Git & GitHub

Libraries

Computer Vision

- OpenCV
- MediaPipe

ML

- PyTorch

System Interaction

- pyautogui
- pynput

Backend (Future)

- FastAPI
-

17. Technology Stack

Layer	Technology
Hand Tracking	MediaPipe
CV Processing	OpenCV
Cursor Control	pyautogui

Layer	Technology
ML Training	PyTorch
Recognition Model	CNN
Stroke Buffer	OpenCV Canvas
Typing Injection	pyautogui/pynput
Future Backend	FastAPI

18. Dataset Details

EMNIST Balanced

Primary dataset for:

- uppercase letters
- lowercase letters
- digits

Features

- 814,255 samples
- balanced classes
- standard OCR benchmark

Optional Dataset

Devanagari Dataset

Future Hindi support.

19. Machine Learning Pipeline

Training Pipeline

1. Download EMNIST
2. Preprocess images
3. Train CNN model
4. Validate model
5. Save trained weights

Runtime Pipeline

1. Extract segmented character crops
 2. Resize to CNN input size
 3. Normalize image
 4. Predict character
 5. Append to running word buffer
 6. Finalise after pause
-

20. CNN Architecture

Recommended Architecture

LeNet-Style CNN

Reason:

- lightweight
- easier debugging
- faster training
- easier FYP explanation
- sufficient for EMNIST

Input

- grayscale image
- 28×28

Layers

- Convolution
 - ReLU
 - MaxPooling
 - Fully Connected
 - Softmax
-

21. Heuristic Segmentation Engine

This is the most important module in the project.

Instead of transformer OCR, AirWrite uses heuristic segmentation.

Segmentation Signals

1. Velocity Drops

Writing speed decreases near boundaries.

2. Pen Lifts

Pinch release indicates possible separation.

3. Stroke Gaps

Spatial gaps suggest character boundaries.

4. X-Axis Clustering

Character groups form distinct horizontal regions.

5. Tiny Pauses

Small hesitations indicate probable transitions.

22. Progressive Prediction Stabilisation

To prevent unstable predictions:

The system accepts predictions only after consistency across multiple frames.

Example:

If same prediction persists for: 5+ frames

then: prediction becomes stable.

This reduces flickering and noisy updates.

23. User Interface Design

Recommended UI Style

Semi-Polished Functional HUD

The system will use:

- clean webcam feed
- fingertip indicator
- mode label
- FPS counter
- current prediction preview
- prediction confidence
- pen down/up indicator

The UI intentionally avoids:

- excessive futuristic graphics
- cluttered overlays
- unnecessary animations

Reason:

- better readability
- easier debugging
- professional FYP presentation

24. Interaction Model

Action	Gesture	Mode
Move Cursor	Index finger	Cursor
Left Click	Thumb-index pinch	Cursor
Right Click	Hold pinch 1s	Cursor
Scroll	Two-finger pinch drag	Cursor
Switch Mode	Open palm hold	Both
Draw Stroke	Pinch + move	Write
Commit Word	1.2s pause	Write
Wipe Current Word	Horizontal full-hand swipe	Write

Action	Gesture	Mode
Delete Previous Word	Horizontal full-hand swipe after commit	Write
Backspace	Keyboard key	Both
Wipe Gesture	Clear current word / delete previous word	Write

25. Risk Analysis

Technical Risks

1. Segmentation Instability

Incorrect boundaries can reduce recognition accuracy.

2. Lighting Sensitivity

Poor lighting affects hand tracking.

3. Motion Blur

Fast writing may reduce tracking quality.

4. Cursor Jitter

Raw landmarks may create unstable movement.

5. Recognition Noise

Continuous writing creates overlapping strokes.

26. Limitations

- depends on webcam quality
 - sensitive to lighting
 - not designed for cursive handwriting
 - moderate writing speed required
 - block-style writing preferred
-

27. Future Enhancements

Future AI Improvements

- Transformer OCR
- CRNN + CTC recognition
- context-aware correction
- multilingual recognition

Future Interaction Improvements

- browser deployment
- mobile deployment
- AR overlays
- eye tracking
- voice integration

Accessibility Expansion

- motor impairment support
 - smart classrooms
 - healthcare systems
-

28. Testing Strategy

Unit Testing

- cursor module
- stroke engine
- segmentation engine
- CNN inference

Integration Testing

- mode switching
- cursor + write pipeline
- typing injection

User Testing

Users perform:

- cursor interaction

- sentence typing
- browser search tasks

Metrics

- character accuracy
 - word accuracy
 - latency
 - FPS
 - Word Error Rate (WER)
 - user fatigue feedback
-

29. Development Roadmap

Month 1 — Core Interaction Engine

Week 1

- webcam setup
- MediaPipe integration
- stroke engine
- fingertip tracking

Week 2

- cursor engine
- click gestures
- adaptive smoothing

Week 3

- CNN training
- EMNIST preprocessing
- inference testing

Week 4

- mode switching
 - continuous stroke buffering
 - progressive prediction pipeline
-

Month 2 — Recognition + Stability

Week 5

- heuristic segmentation engine
- candidate extraction

Week 6

- prediction stabilisation
- threshold tuning
- latency optimisation

Week 7

- benchmarking
- confusion matrix
- WER analysis

Week 8

- optional FastAPI groundwork
 - architecture cleanup
-

Month 3 — Finalisation

Week 9

- user testing
- accessibility testing

Week 10

- report writing
- architecture diagrams
- methodology section

Week 11

- demo video
- GitHub documentation
- FYP poster

Week 12

- final polishing
- edge-case handling

- optional Hindi prototype
-

30. Deployment Plan

Current Deployment

Desktop-based Python application.

Future Deployment

- FastAPI backend
- browser frontend
- possible React integration

Browser deployment remains future scope because:

- pyautogui requires desktop-level permissions
 - OS-level typing is desktop dependent
-

31. Comparison with GhostPen

GhostPen	AirWrite
Gesture classification	Continuous word recognition
LSTM	CNN
Custom dataset	EMNIST
Sequence windows	Stroke buffers
No cursor control	Full cursor interaction
App-local output	OS-level typing

32. Expected Outcomes

The final system should:

- control the system cursor
- perform click interactions
- recognise air-written words

- inject words into any application
 - support touchless deletion gestures
 - operate in real time
 - function using only a webcam
 - demonstrate accessibility-focused interaction
-

33. Research Contribution

AirWrite contributes toward:

- touchless operating system interaction
 - webcam-based assistive technology
 - continuous air-written word recognition
 - heuristic online segmentation systems
 - accessibility-focused HCI research
-

34. Conclusion

AirWrite is a next-generation touchless interaction system combining gesture-based cursor control, continuous air handwriting, heuristic segmentation, and AI-based character recognition into a unified accessibility-focused platform.

By avoiding specialised hardware and enabling OS-level interaction using only a webcam, AirWrite creates a realistic and scalable Human-Computer Interaction system with strong academic novelty and real-world usability.

The project represents a practical middle ground between traditional gesture systems and advanced online handwriting recognition systems while remaining achievable within an undergraduate Final Year Project scope.

35. References

1. MediaPipe Hands Documentation
 2. OpenCV Documentation
 3. EMNIST Research Paper — Cohen et al. 2017
 4. pyautogui Documentation
 5. Human-Computer Interaction Research Papers
 6. AirDraw Gesture Projects
 7. GhostPen Project Repository
-

36. Appendix

Proposed Folder Structure

AirWrite/ | ├── core/ | ├── stroke_engine.py | ├── cursor_engine.py | ├──
segmentation_engine.py | ├── gesture_engine.py | ├── ml/ | ├── training/ | ├── datasets/ |
| ├── models/ | ├── inference/ | ├── utils/ | ├── smoothing.py | ├── hud.py | ├──
helpers.py | ├── docs/ | ├── tests/ | ├── README.md

Initial Build Order

Phase 1

stroke_engine.py

Phase 2

cursor_engine.py

Phase 3

cnn_training.py

Phase 4

segmentation_engine.py

Phase 5

full_pipeline.py

END OF DOCUMENT